

4.1 The Eight-Point Algorithm Still, the assumption that we can have the Fundamental matrix, which is defined by a matrix product of the camera parameters, seems rather large. However, it is possible to estimate the Fundamental matrix given two images of the same scene and without knowing the extrinsic or intrinsic parameters of the camera. The method we discuss for doing so is known as the Eight-Point Algorithm, which was proposed by Longuet-Higgins in 1981 and extended by Hartley in 1995. As the title

Show more ▾

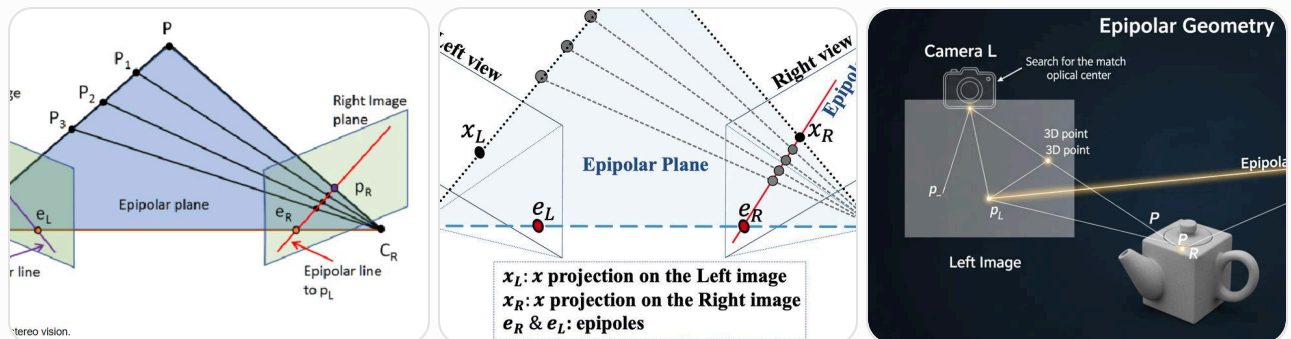
Đoạn này đang đi theo một flow rất tự nhiên:

Corresponding points → Epipolar constraint → Linear system $Wf = 0$ → SVD → rank-2 correction → Normalization → Fundamental Matrix thực tế

Mình đi đúng theo luồng đó.

1. Bài toán: tìm Fundamental Matrix từ 2 ảnh

Ta có hai ảnh của cùng một scene:



Giả sử ta tìm được các cặp điểm tương ứng:

$$p_i = (u_i, v_i, 1)$$

trong ảnh 1 và

$$p'_i = (u'_i, v'_i, 1)$$

trong ảnh 2.

Điểm quan trọng:

Ta không biết camera intrinsic K, K' , cũng không biết extrinsic R, T .

Nhưng ta vẫn muốn tìm:

$$F$$

Fundamental Matrix.

2. Ta đã biết Epipolar Constraint

Từ phần trước:

$$p_i^T F p'_i = 0$$

Đây là **epipolar constraint**.

Nó nói rằng:

Nếu p_i và p'_i thực sự là cùng một điểm 3D được nhìn từ hai camera, thì chúng phải thỏa mãn phương trình trên.

Mỗi cặp correspondence cho ta **một phương trình**.

3. Biến phương trình thành dạng tuyến tính

Viết:

$$F = \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix}$$

và:

$$p_i = \begin{bmatrix} u_i \\ v_i \\ 1 \end{bmatrix}, \quad p'_i = \begin{bmatrix} u'_i \\ v'_i \\ 1 \end{bmatrix}$$

Khi khai triển:

$$p_i^T F p'_i = 0$$

ta được:

$$u_i u'_i F_{11} + u_i v'_i F_{12} + u_i F_{13} + v_i u'_i F_{21} + v_i v'_i F_{22} + v_i F_{23} + u'_i F_{31} + v'_i F_{32} + F_{33} =$$



Nhìn kỹ:

$$\underbrace{[u_i u'_i, u_i v'_i, u_i, v_i u'_i, v_i v'_i, v_i, u'_i, v'_i, 1]}_{1 \times 9}$$

nhân với:

$$\underbrace{\begin{bmatrix} F_{11} \\ F_{12} \\ F_{13} \\ F_{21} \\ F_{22} \\ F_{23} \\ F_{31} \\ F_{32} \\ F_{33} \end{bmatrix}}_{9 \times 1} = 0$$

4. Gom nhiều correspondence $\rightarrow Wf = 0$

Với một correspondence:

$$w_i f = 0$$

Với N correspondence:

$$Wf = 0$$

trong đó:

$$W = \begin{bmatrix} w_1 \\ w_2 \\ \vdots \\ w_N \end{bmatrix}$$

có kích thước:

$$N \times 9$$

và:

$$f = [F_{11} \quad F_{12} \quad \cdots \quad F_{33}]^T$$

là 9 phần tử của Fundamental Matrix.

5. Tại sao cần ít nhất 8 điểm?

Một 3×3 matrix có:

$$9$$

unknowns.

Nhưng F chỉ xác định **up to scale**.

Ví dụ:

$$F$$

và:

$$2F$$

mô tả cùng một epipolar geometry vì:

$$p^T (2F) p' = 2p^T F p' = 0$$

Do đó thực tế chỉ có:

$$9 - 1 = 8$$

degrees of freedom.

Mỗi correspondence cho **1 constraint**.

Vì vậy cần ít nhất:

8 correspondences

Đây là nguồn gốc của tên:

Eight-Point Algorithm

6. Nhưng tại sao không giải $Wf = 0$ bình thường?

Đây là điểm rất quan trọng.

Ta có:

$$Wf = 0$$

Nếu W có full rank 9 thì nghiệm duy nhất là:

$$f = 0$$

mà ta không muốn.

Vì vậy ta cần tìm **non-zero vector** f nằm trong null space của W .

SVD là cách tự nhiên để làm việc này.

7. SVD của W

Ta phân rã:

$$W = U\Sigma V^T$$

Vector f tương ứng với **singular value nhỏ nhất** của W .

Nói đơn giản:

Ta tìm f sao cho Wf gần 0 nhất.

Tức là:

$f = \text{right singular vector của } W \text{ ứng với singular value nhỏ nhất}$

Sau đó reshape 9 phần tử thành:

$$\hat{F} = \begin{bmatrix} F_{11} & F_{12} & F_{13} \\ F_{21} & F_{22} & F_{23} \\ F_{31} & F_{32} & F_{33} \end{bmatrix}$$

Đây là ước lượng ban đầu.

8. Vấn đề: $\hat{F}$ có thể không phải Fundamental Matrix hợp lệ

Fundamental Matrix thật phải thỏa:

$$\text{rank}(F) = 2$$

Hay tương đương:

$$\det(F) = 0$$

Nhưng do noise trong correspondence, SVD có thể cho:

$$\hat{F}$$

có:

$$\text{rank}(\hat{F}) = 3$$

Ví dụ:

$$\hat{F} = \begin{bmatrix} * & * & * \\ * & * & * \\ * & * & * \end{bmatrix}$$

có 3 singular values khác 0:

$$\sigma_1, \sigma_2, \sigma_3$$

Trong khi Fundamental Matrix thật phải có:

$$\sigma_3 = 0$$

9. Rank-2 approximation

Ta SVD tiếp:

$$\hat{F} = U\Sigma V^T$$

với:

$$\Sigma = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & \sigma_3 \end{bmatrix}$$

Ta ép:

$$\sigma_3 = 0$$

để có:

$$\Sigma' = \begin{bmatrix} \sigma_1 & 0 & 0 \\ 0 & \sigma_2 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

Sau đó:

$$F = U\Sigma'V^T$$

Đây chính là **best rank-2 approximation** của $\hat{F}$.

Trực giác:

SVD tìm F tốt nhất từ dữ liệu → sau đó sửa F để nó tuân thủ constraint hình học bắt buộc: rank 2.

10. Vậy Standard Eight-Point Algorithm hoàn chỉnh là gì?

Flow:

Two images

↓

Find corresponding points

↓



```

p_i ↔ p'_i
↓
Epipolar constraint
p_i^T F p'_i = 0
↓
Expand equation
↓
wf = 0
↓
SVD(w)
↓
f = singular vector corresponding
    to smallest singular value
↓
reshape f → F_hat
↓
SVD(F_hat)
↓
set smallest singular value = 0
↓
F = U Σ' V^T
↓
Fundamental Matrix F

```

Đó là **Eight-Point Algorithm**.

11. Nhưng Standard Eight-Point Algorithm có vấn đề rất lớn

Đây là phần **4.2 Normalized Eight-Point Algorithm**.

Giả sử image có resolution:

$$4000 \times 3000$$

Một point có thể là:

$$p = (1832, 1023, 1)$$

Các giá trị u, v rất lớn.

Khi tạo $\mathbf{W}$, ta có các term:

$$uu', \quad uv', \quad vu', \quad vv'$$

Ví dụ:

$$1832 \times 1700 \approx 3.1 \times 10^6$$

trong khi một số phần tử khác chỉ là:

$$u \approx 1832$$

hoặc:

$$1$$

=> các phần tử của W có **scale rất khác nhau**.

Điều này khiến:

W bị ill-conditioned

và SVD trở nên nhạy với numerical error.

12. Đặc biệt tệ khi các điểm nằm trong một vùng nhỏ

Ví dụ ảnh là:



Các point đều gần nhau.

Ví dụ:

$$(1830, 1020), (1832, 1023), (1828, 1021), \dots$$

Các row của W sẽ rất giống nhau.

Kết quả:

W có condition number rất xấu $\rightarrow$ việc tìm null space bằng SVD kém ổn định.

Đây là lý do cần **normalization**.

13. Normalized Eight-Point Algorithm làm gì?

Trước khi xây dựng W , ta **normalize coordinates**.

Có hai việc:

Bước 1: Translation

Đưa centroid của các points về origin.

Nếu centroid là:

$$\bar{p} = (\bar{u}, \bar{v})$$

thì:

$$u'_i = u_i - \bar{u}$$

$$v'_i = v_i - \bar{v}$$

Sau transformation:

Original

```
x x x
x x x
x x x
```

$\rightarrow$

Normalized

```
x x x
x x x
x x x
```

(0, 0)

Centroid nằm tại:

$$(0, 0)$$

14. Bước 2: Scaling

Sau khi đưa centroid về origin, ta scale points sao cho:

$$\text{mean square distance} = 2$$

Tức:

$$\frac{1}{N} \sum_i \|q_i\|^2 = 2$$

hay scale factor được chọn dựa trên:

$$\sqrt{\frac{2N}{\sum_i \|x_i - \bar{x}\|^2}}$$

Mục đích không phải vì **2** là một con số thần kỳ.

Mục đích là:

đưa coordinates về một scale hợp lý và đồng nhất.

15. Gói translation + scaling thành matrix T

Ta viết:

$$q_i = T p_i$$

và với image thứ hai:

$$q'_i = T' p'_i$$

Trong đó T, T' là normalization matrices.

Ví dụ về mặt concept:

$$T = \begin{bmatrix} s & 0 & -s\bar{u} \\ 0 & s & -s\bar{v} \\ 0 & 0 & 1 \end{bmatrix}$$

Nó thực hiện:

1. translate centroid $\rightarrow$ origin

2. scale coordinates

16. Sau normalization, chạy Eight-Point Algorithm bình thường

Ta có:

$$q_i = T p_i$$

$$q'_i = T' p'_i$$

Bây giờ dùng:

$$q_i^T F_q q'_i = 0$$

và xây dựng:

$$W_q f_q = 0$$

Sau đó:

SVD(W_q)

↓

F_q_hat

↓

rank-2 enforcement

↓

F_q



Điểm quan trọng:

$$F_q$$

là Fundamental Matrix **trong normalized coordinate system**.

Nó chưa phải F dùng trực tiếp với pixel coordinates ban đầu.

17. De-normalization

Ta có:

$$q = T p$$

và:

$$q' = T' p'$$

Epipolar constraint trong normalized space:

$$q^T F_q q' = 0$$

thay vào:

$$(Tp)^T F_q (T' p') = 0$$

Ta được:

$$p^T T^T F_q T' p' = 0$$

Do đó:

$$F = T^T F_q T'$$

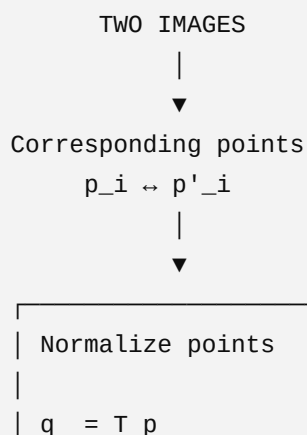
Tùy convention đặt ảnh 1/ảnh 2 và cách định nghĩa T, T' , công thức trong tài liệu có thể được viết:

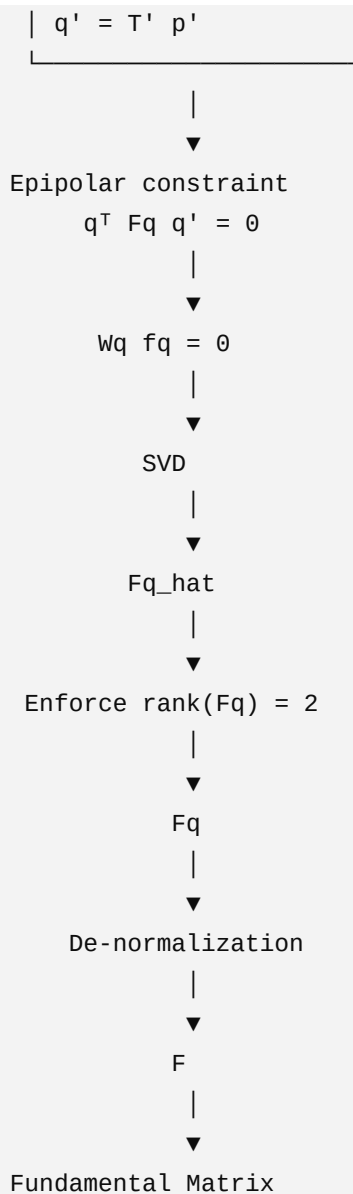
$$F = T'^T F_q T$$

Điều quan trọng là **cấu trúc biến đổi**, không nên học thuộc máy móc công thức nếu chưa thống nhất p, p' tương ứng với camera nào.

18. Toàn bộ Normalized Eight-Point Algorithm

Có thể nhớ bằng flow này:





19. Tại sao Normalization lại giúp?

Đây là ý quan trọng nhất của phần 4.2.

Không normalize:

$$p = (1832, 1023, 1)$$

→ coordinates lớn

→ W có scale rất không đồng đều

→ W ill-conditioned

→ SVD nhạy với numerical error

→ F kém chính xác.

Normalize:

$$p \rightarrow q$$

sao cho:

- centroid $\approx (0, 0)$
- average distance $\approx \sqrt{2}$

→ coordinates có scale hợp lý

→ W tốt hơn về numerical conditioning

→ SVD ổn định hơn

→ F chính xác hơn.

Đó là lý do trong thực tế:

Normalized Eight-Point Algorithm $\gg$ Standard Eight-Point Algorithm

về numerical stability.

20. Một điểm rất dễ nhầm: "8 points" không có nghĩa là thực tế chỉ dùng 8 points

Tên vẫn là **Eight-Point Algorithm**, nhưng ta hoàn toàn có thể dùng:

$$N = 20, 50, 100, 1000...$$

Khi đó:

$$W \in \mathbb{R}^{N \times 9}$$

Mỗi correspondence thêm một constraint:

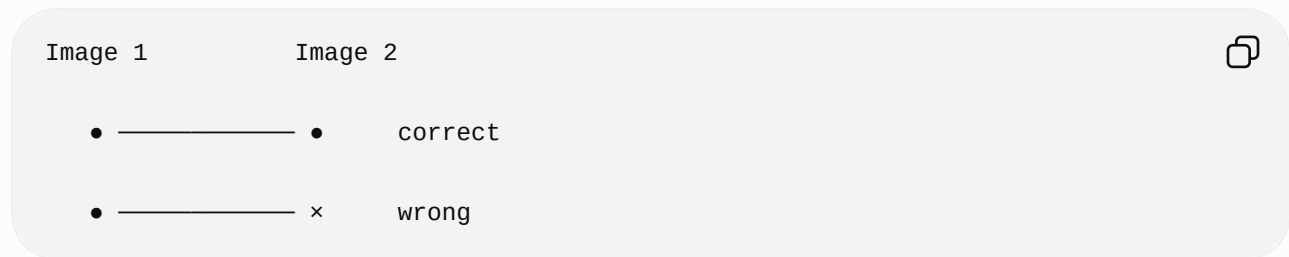
$$w_i f \approx 0$$

Trong dữ liệu thực tế có noise, dùng nhiều points giúp ước lượng robust hơn.

21. Nhưng nếu correspondence có outlier thì sao?

Đây là bước mà đoạn tài liệu này **chưa đi sâu**.

Ví dụ:



Nếu có correspondence sai, Eight-Point Algorithm thuần túy có thể bị ảnh hưởng mạnh.

Thực tế thường dùng:

RANSAC + Normalized Eight-Point Algorithm

Flow phổ biến:



Đây mới gần với pipeline Computer Vision thực tế.

22. Bức tranh lớn: tại sao Eight-Point Algorithm quan trọng?

Nó giải quyết một bài toán rất mạnh:

Chỉ từ hai ảnh và các điểm tương ứng, không cần biết camera calibration hay camera pose, ta có thể estimate Fundamental Matrix.

Tức:

$$\boxed{\text{Correspondences} \rightarrow F}$$

Sau khi có F , ta có thể suy ra **epipolar geometry**:

$$l' = Fp$$

hoặc:

$$l = F^T p'$$

Từ đó constraint việc tìm correspondence trở nên mạnh hơn.

Và nếu biết intrinsic parameters K, K' , ta có thể đi tiếp:

$$\boxed{F \rightarrow E}$$

với:

$$E = K'^T F K$$

rồi từ E :

$$\boxed{E \rightarrow R, T}$$

và từ đó tiến tới:

$$\boxed{R, T + \text{correspondences} \rightarrow 3D \text{ reconstruction}}$$

Đây chính là cầu nối từ **2D image correspondence** → **camera geometry** → **3D vision**.

Mental model nên nhớ

Nếu học để đi vào **3D Vision / Robotics**, mình khuyên bạn nhớ phần này thành 5 tầng:

$$\boxed{1. \text{Correspondence}}$$


2. Epipolar constraint $p^T F p' = 0$



3. Estimate F using SVD



4. Enforce $\text{rank}(F) = 2$



5. Normalize first for numerical stability

Và **Normalized Eight-Point Algorithm** thực chất chỉ là:

Normalize → Eight-Point → Rank-2 → De-normalize.

Đây là công thức tinh thần quan trọng nhất của section 4.1–4.2.



ChatGPT can make mistakes. Check important info.